

Teoría de Algoritmos
(TB024) - Curso Echevarría

Trabajo Práctico 2

Fecha de entrega:
21 de junio de 2026

Integrantes:

Brizuela Sebastian

105288

Chen Nicolas

105907

Janon Santiago Ignacio

106079

Lanseros Nicolas

111762

Ordoñez Joaquin

108865

ÍNDICE

Problema 1 - Programación lineal:	4
1. Introducción:	4
2. Definición del problema:	4
3. Análisis del problema:	4
3.1 Variable de decisión y constantes del problema	4
3.2. Función objetivo	5
3.3 Restricciones	5
4. Algoritmo:	5
4.1. Formulación matemática:	6
4.2. Pseudocódigo:	6
4.4. Implementación:	7
4.5. Solución óptima obtenida	8
4.5.1. Demostración de optimalidad	8
4.5.2. Complejidad temporal	9
4.5.3. Complejidad espacial	10
5. Mediciones y validaciones empíricas	10
6. Conclusión	10
Problema 2 - Redes de flujo:	11
1. Introducción:	11
2. Definición del problema:	11
3. Análisis del problema:	12
4. Algoritmo:	12
4.1. Pseudocódigo:	12
4.4. Implementación:	13
4.5. Complejidad:	14
4.1. Complejidad Temporal:	14
4.2. Complejidad Espacial:	15
5. Ejemplo de ejecución y seguimiento:	16
Paso 1: Construcción de la Red de Flujo (G)	16
Paso 2: Traza del Algoritmo de Flujo Máximo (Edmonds-Karp)	17
Paso 3: Estado Final de las Variables Internas	17
Paso 4: Verificación y Reconstrucción del Resultado	17
6. Mediciones y validaciones empíricas:	18
6.1. Registro de Tiempos de Ejecución	18
6.2. Análisis del Gráfico y Ajuste Curvilíneo	19
Problema 3 - Algoritmos de aproximación:	20
1. Introducción:	20
2. Definición del problema y contraejemplo:	20

2.1. El enfoque “ciego” y su caso de falla:	20
2.2. Seguimiento de la instancia:	21
2.3. Demostración de la violación de garantía:	21
3. Análisis del problema, supuestos y limitaciones:	21
3.1. Análisis de la falla y estrategia de mitigación:	21
3.2. Supuestos y precondiciones:	22
3.3. Comportamiento en casos límite:	22
4. Diseño del Algoritmo:	22
4.1. Descripción de la solución:	22
4.2. Demostración matemática de la garantía:	22
4.3. Pseudocódigo:	24
4.4. Estructuras de datos:	24
4.5. Implementación:	24
5. Seguimiento: Ejemplo de ejecución:	25
6. Análisis de Complejidad:	25
7. Set de datos:	26
8. Validaciones empíricas:	26
9. Conclusión:	28
Problema 4 - Algoritmos aleatorios:	29
1. Introducción:	29
2. Definición del problema:	29
3. Análisis del problema:	29
4. Algoritmo:	29
4.1. Formulación del algoritmo aleatorio:	29
4.2. Variable Indicadora:	30
4.3. Cálculo del valor esperado:	30
4.4. Estructuras utilizadas:	30
4.5. Pseudocódigo:	31
4.6. Implementación:	31
4.7. Complejidad:	32
4.7.1. Complejidad Temporal:	32
5. Ejemplo de ejecución y seguimiento:	32
6. Mediciones:	34
6.1. Metodología de pruebas:	34
6.2. Gráfico comparativo:	34
Imagen: Promedio vs Intentos:	35
7. Conclusión:	35

Problema 1 - Programación lineal:

1. Introducción:

En la presente sección se realizará un análisis y se describirá la solución al problema de “las concesiones de los espacios publicitarios en las paradas de colectivos en un municipio”. Para ello, se describirá un algoritmo de programación lineal que encuentre la solución óptima al problema, analizando cuál es su función objetivo, y cuáles son las restricciones al problema.

2. Definición del problema:

La empresa Concesiones Argentina 2000 SRL tiene la concesión de los espacios publicitarios en las paradas de colectivos de un municipio, y tiene oferta de distintos productos para el próximo mes:

- Cliente A: ofrece USD 50000 por ocupar 30 paradas
- Cliente B: ofrece USD 10000 por ocupar 80 paradas o USD 1200000 por 120 paradas (sólo se puede tomar una)
- Cliente C: ofrece USD 100000 por ocupar 75 paradas
- Cliente D: ofrece USD 80000 por ocupar 50 paradas
- Cliente E: ofrece USD 5000 por ocupar 2 paradas
- Cliente F: ofrece USD 40000 por ocupar 20 paradas
- Cliente G: ofrece USD 90000 por ocupar 100 paradas

Por ser competidores directos, no se puede aceptar la oferta de los clientes A y D en simultáneo. Sólo se puede tomar una de ellas.

El objetivo es determinar a qué clientes se concesionará la publicidad de los espacios, para maximizar el beneficio total.

3. Análisis del problema:

Para poder aplicar la técnica de programación lineal, es necesario modelar el problema matemáticamente para que un software adecuado lo pueda resolver. Específicamente, se requiere una/s variables de decisión, una función objetivo y un conjunto de restricciones

3.1 Variable de decisión y constantes del problema

Para este problema, se define la siguiente variable de decisión:

- $Y_{i,k}$: variable binaria que vale 1 si se acepta la oferta número k del cliente i.

Conocer el número de la oferta aceptada es fundamental para resolver la restricción planteada en el problema, en relación al cliente B.

Además, en el modelo se definen las siguientes constantes:

- $V_{i,k}$: valor de la oferta número k del cliente i.
- $C_{i,k}$: cantidad de paradas de la oferta número k del cliente i.

3.2. Función objetivo

En este caso, el objetivo es claro: se busca maximizar la ganancia total. Es decir, se quiere saber cuáles ofertas son aceptadas, y el beneficio que se obtiene de cada una de ellas. Matemáticamente, esto se puede escribir de la siguiente forma:

$$Z = \max(\sum_{i=1}^7 \sum_{k \in D} v_{i,k} \cdot Y_{i,k}),$$

siendo D el conjunto que representa la cantidad de ofertas de cada cliente.

Además, en el modelo se definen las siguientes constantes:

- $V_{i,k}$: valor de la oferta número k del cliente i.
- $C_{i,k}$: cantidad de paradas de la oferta número k del cliente i.

3.3 Restricciones

El problema cuenta con dos restricciones definidas explícitamente en el enunciado del problema:

1. Solamente se puede elegir una oferta del cliente B. Aquí es dónde resulta importante saber el número de la oferta del cliente que se está aceptando: la restricción se transforma en:

$$Y_{b,1} + Y_{b,2} \leq 1.$$

Esto obliga al modelo a elegir sólo una de las dos posibilidades.

2. No se puede elegir simultáneamente las ofertas de los clientes A y D. Matemáticamente, esta limitación se escribe de una forma similar a la anterior:

$$Y_{a,1} + Y_{d,1} \leq 1.$$

Tal cual se describió en el caso anterior, esta inecuación obliga al modelo a elegir sólo uno de los dos caminos.

3. La última inecuación, en cambio, no se encuentra definida formalmente en el problema, pero es importante para que la solución obtenida sea válida: la cantidad de paradas que tiene la solución no puede ser mayor a 200 (la suma total). Esto se puede escribir de la siguiente forma:

$$\sum_{i=1}^7 \sum_{k \in D} Y_{i,k} \cdot c_{i,k} \leq 200.$$

4. Algoritmo:

En base al análisis previo, se propone un algoritmo en Python, que resuelve el problema mediante el uso de la biblioteca PULP. El programa recibe los datos de un archivo llamado "ofertas.csv", en donde cada propuesta se escribe así:

nombre_cliente, numero_oferta, valor_oferta, cantidad_paradas.

Luego de cargar los datos, el programa define las variables, la función objetivo y las restricciones utilizando herramientas que provee la biblioteca. Finalmente, una vez que termina con la ejecución, imprime el resultado óptimo obtenido en el archivo "Archivo_resultados.csv"

4.1. Formulación matemática:

Sea $E = \{e_1, e_2, \dots, e_n\}$ el conjunto de los n ofertas, en donde cada e se compone de 4 elementos (i, k, v, c) , en donde:

- i : nombre del cliente
- k : identificador de la oferta del cliente i
- $v_{i,k}$: Valor económico de la oferta
- $c_{i,k}$: cantidad de paradas requeridas por la oferta.

Sea $Y_{i,k} \in \{0, 1\}$ la variable de decisión que indica si se acepta la oferta k del cliente i .

Función Objetivo:

$$\max(Z) = \sum_{k \in D} v_{i,k} \cdot Y_{i,k}$$

Sujeto a las restricciones:

1. Capacidad: $\sum c_{i,k} \cdot Y_{i,k} \leq 200$
2. Exclusividad (Cliente B): $Y_{B,1} + Y_{B,2} \leq 1$
3. Competencia (A y D): $Y_{A,1} + Y_{D,1} \leq 1$

4.2. Pseudocódigo:

Como se mencionó previamente, el ejercicio se resuelve a partir del uso de la biblioteca PULP. Un aspecto importante es que los datos se guardan en un diccionario. Para ello, la clave que identifica a cada oferta es: `(nombre_cliente, numero_oferta)`

```
// Función que resuelve el modelo
Función resolverModelo(dict_ofertas):
    // Inicio el modelo de PL
    prob = LpProblem("Maximizacion_Beneficios_Publicidad", Maximizar)
    indices = dict_ofertas.claves()
    // Declaro las variables y funcion objetivo
    Y = Variable("ofertas", indices, binaria)
    prob += (Sumar(Y[i] * dict_ofertas[i]["valor"] para i en indices, "Z")

    // Defino las restricciones
    // Solo una oferta de B
    prob += (Sumar(Y[i] para i en indices si i[0] == "B") <= 1
    // Solo una oferta de A o D
    prob += (Sumar(Y[i]* para i en índices si i[0] == "A") + (Y[i] para i en indices si
i[0] == "D")) <= 1
    // Hasta 200 publicidades
    prob += (Sumar(Y[i] * dict_ofertas[i]["cantidad"] para i en indices) <= 200

    prob.escribir("Problema_optimizacion_lineal.lp")
    status = prob.resolver()

    // Extraigo los resultados
    resultados = dict()
    return resultados
```

4.4. Implementación:

```
def cargar_archivo(ruta):
    """Carga las ofertas y retorna un diccionario con los datos."""
    ofertas = {}
    try:
        with open(ruta, "r") as archivo:
            for linea in archivo:
                valores = linea.replace(',', ' ').split()
                if len(valores) < 4:
                    continue
                clave = (valores[0].strip(), valores[1].strip())
                ofertas[clave] = {
                    "valor": int(valores[2].strip()),
                    "cantidad": int(valores[3].strip())
                }
    except FileNotFoundError:
        print(f"Error: No se encontró el archivo {ruta}", file=sys.stderr)
        sys.exit(1)

    return ofertas

def resolver_modelo(dict_ofertas, ruta_modelo):
    """
    Define y resuelve el modelo de programación lineal.
    Retorna un diccionario con los resultados matemáticos puros.
    """
    prob = LpProblem("Maximizacion_Beneficios_Publicidad", LpMaximize)
    indices = list(dict_ofertas.keys())

    # Variables de decisión
    Y = LpVariable.dicts("Oferta", indices, cat=LpBinary)

    # Función objetivo
    prob += lpSum(Y[i] * dict_ofertas[i]["valor"] for i in indices), "Beneficio_Total"

    # Restricciones
    # 1. Solo se puede aceptar una oferta del cliente B (B = 1)
    prob += (lpSum(Y[i] for i in indices if i[0] == "B")) <= 1
    # 2. Solo se pueden aceptar la oferta del cliente A o del cliente D, no ambas
    prob += (lpSum(Y[i] for i in indices if i[0] == "A") + lpSum(Y[i] for i in indices if
i[0] == "D")) <= 1
    # 3. la cantidad de paradas vendidas no puede ser mayor a 200
    prob += lpSum(Y[i] * dict_ofertas[i]["cantidad"] for i in indices) <= 200

    # Asegurar que exista la carpeta 'model' antes de guardar
    os.makedirs(os.path.dirname(ruta_modelo), exist_ok=True)
    prob.writeLP(ruta_modelo)

    # Resolución silenciosa (msg=False evita que PuLP ensucie la consola)
    start_time = time.perf_counter()
```

```

status = prob.solve(PULP_CBC_CMD(msg=False))
end_time = time.perf_counter()

# Empaquetar resultados
resultados = {
    "estado": LpStatus[status],
    "tiempo_ejecucion": end_time - start_time,
    "maximo": 0,
    "paradas_totales": 0,
    "ofertas_aceptadas": []
}

if LpStatus[status] == "Optimal":
    resultados["maximo"] = value(prob.objective)
    for i in indices:
        if Y[i].varValue == 1:
            resultados["ofertas_aceptadas"].append({
                "cliente": i[0],
                "oferta": i[1],
                "valor": dict_ofertas[i]["valor"],
                "cantidad": dict_ofertas[i]["cantidad"]
            })
            resultados["paradas_totales"] += dict_ofertas[i]["cantidad"]

return resultados

```

4.5. Solución óptima obtenida

Luego de finalizar la ejecución del programa, el algoritmo obtiene la siguiente solución (que escribe en el archivo mencionado:

- Cliente: A, Numero oferta: 1, Valor: 50000, Paradas: 30
- Cliente: B, Numero oferta: 1, Valor: 100000, Paradas: 80
- Cliente: C, Numero oferta: 1, Valor: 100000, Paradas: 75
- Cliente: E, Numero oferta: 1, Valor: 5000, Paradas: 2
- Máximo obtenido: 255000.0 Cantidad total de paradas: 187

En la siguiente sección, se analizará cómo se puede demostrar que es la solución óptima.

4.5.1. Demostración de optimalidad

Para analizar que el resultado obtenido es correcto, el análisis se divide en dos etapas:

Verificación de Factibilidad

- La primera restricción establecía que sólo se podía elegir una de las dos ofertas del cliente B. Esta limitación se cumple; el algoritmo sólo agrega la primera oferta.
- La segunda limitación obligaba a elegir la oferta del cliente A, o la del cliente D, pero no ambas. En este caso, la solución sólo incluye la propuesta del cliente A, por lo que es válida en ese sentido.

- La última restricción limitaba a que la cantidad de paradas vendidas no supere la cantidad total de las mismas. Esto también se cumple, ya que la cantidad de paradas de la solución (187) no supera la totalidad de las paradas (200).

Realizado este análisis, se puede asegurar que la solución es válida en términos de las restricciones del problema. En otras palabras, las restricciones planteadas en el modelo son fuertes y ayudan a encontrar una respuesta válida al problema.

Garantía de optimalidad

La optimalidad del valor obtenido está garantizada por el solver invocado mediante la biblioteca Pulp. Dicho solver utiliza la técnica Branch and Bound para resolver problemas de programación lineal entera, realizando una búsqueda sistemática en el árbol de soluciones. El proceso, en síntesis, funciona de la siguiente manera:

- Relajación lineal: de esta forma, se calculan cotas que sirve como referencia teórica
- Podas: se descartan aquellas soluciones que, en términos matemáticos, no tienen posibilidad de superar el mejor beneficio hallado hasta el momento.
- Certificado final: al finalizar el proceso, el algoritmo devuelve un valor de estado `LpStatusOptimal`, lo que garantiza que, bajo esas restricciones, no existe una solución mejor que la obtenida.

4.5.2. Complejidad temporal

El análisis de la complejidad temporal del algoritmo se divide en dos partes: la modelación del problema y su resolución.

Modelación del problema

La construcción de la estructura del problema (es decir, la creación de variables y la definición de restricciones) crecen en forma lineal en relación a la cantidad “n” de ofertas. Cada variable, y cada restricción, se define una sola vez para cada oferta recibida, por lo que la complejidad de esta parte del problema es $O(n)$.

Resolución del modelo

Esta parte es la que define la complejidad del problema. Al ser un problema con variables enteras (específicamente, se trata de variables binarias), el solver utiliza Branch and Bound para resolverlo, lo que significa que se recorre sistemáticamente el espacio de soluciones. En el peor de los casos, dicho espacio de soluciones crece exponencialmente por cada variable de decisión. Esto quiere decir que la complejidad de la resolución del modelo es exponencial: $O(2^n)$. Sin embargo, gracias a las podas descritas anteriormente, el solver descarta gran parte del árbol de búsqueda, lo que significa que, en la práctica, resuelve el problema en mucho menos tiempo.

4.5.3. Complejidad espacial

Para poder resolver el problema, el programa necesita almacenar en memoria las “n” variables de decisión definidas, y las “m” restricciones. Por lo tanto, la complejidad espacial de esta parte del problema es $O(n + m)$.

En cuanto al proceso de resolución, durante la exploración del árbol de búsqueda, el algoritmo almacena los nodos activos en una pila. Si definimos a “d” como el nivel de profundidad del árbol, la estructura en memoria requerirá un espacio de $O(n \cdot d)$. En el peor de los casos, el nivel de profundidad “d” puede ser equivalente al número de variables, alcanzando una complejidad teórica de $O(n^2)$.

5. Mediciones y validaciones empíricas

Para evaluar el desempeño del algoritmo, se realizaron mediciones con distintos archivos de entrada. Cada archivo corresponde a un problema con distinta cantidad de ofertas (con $n=10$, $n=25$ y $n=50$, respectivamente). El objetivo fue validar la correctitud de la solución frente a las restricciones, así como confirmar el comportamiento temporal del solver.

Tanto los archivos, como las soluciones óptimas, se adjuntan en la carpeta que contiene el código del programa

Los resultados obtenidos de las mediciones fueron los siguientes:

Cantidad de ofertas	Espacio de búsqueda	Tiempo de ejecución (segundos)	Estado
10	1024	0.025403	Óptimo
25	33554432	0,048138	Óptimo
50	2^{50}	0,0409777	Óptimo

Análisis de los resultados

Los resultados son un reflejo del planteo teórico realizado anteriormente. Aunque el árbol de búsqueda crece en forma exponencial, el tiempo de ejecución real se mantiene muy por debajo de su cota teórica. Esto confirma que el Branch and Bound, mediante el uso de las podas mencionadas, logra evitar la exploración de ramas que no van a mejorar el óptimo global, haciendo que el software sea muy eficiente para este rango de variables.

6. Conclusión

En esta sección del informe se ha planteado un modelo de programación lineal que resuelve de forma eficiente el problema de las concesiones de los espacios publicitarios de las paradas de colectivos. Se han explicado las restricciones que tiene el problema, y se ha demostrado que dichas restricciones, utilizando un software adecuado para este tipo de problemas, son conducentes a encontrar la solución óptima del problema. Además, se ha validado empíricamente que, frente a distintos problemas, el solver mantiene un tiempo de respuesta mucho menor a la complejidad teórica establecida (de orden exponencial).

Entonces, a modo de conclusión, se puede afirmar lo siguiente: el modelo planteado es sólido y, a través de un software eficiente, permite encontrar una respuesta rápidamente al problema.

Problema 2 - Redes de flujo:

1. Introducción:

En esta sección vamos a resolver un problema de asignación con restricciones utilizando el modelo de Redes de Flujo. El enunciado nos presenta un caso práctico sobre cómo organizar antenas WAN para que sean tolerantes a fallas, con el objetivo de ver cómo podemos traducir un problema con varias condiciones cruzadas a un grafo dirigido y resolverlo de forma eficiente.

Para cumplir con lo pedido, modelamos el problema como un grafo bipartito donde las restricciones se transforman en capacidades de las aristas. Para encontrar la solución, aplicamos el algoritmo de Edmonds-Karp (que es la variante de Ford-Fulkerson que usa BFS). Elegimos esta opción porque nos asegura que el algoritmo va a terminar en un tiempo polinomial, sin importar qué tan grandes sean los números que nos pasen como parámetros. En el informe vamos a explicar cómo diseñamos el grafo, el pseudocódigo, el análisis de complejidad y las pruebas con distintos datos.

2. Definición del problema:

El enunciado nos pide armar un algoritmo que encuentre un conjunto de backup de tamaño k para cada una de las n antenas de una red, cumpliendo las siguientes condiciones:

- Entrada del problema:
 1. n : Cantidad total de antenas.
 2. D : Distancia máxima permitida para que una antena pueda ser backup de otra.
 3. k : Cuántas antenas de backup necesita tener cada antena.
 4. b : El límite de cuántas veces una misma antena puede aparecer en los conjuntos de backup de las demás.
 5. d : Una matriz de $n \times n$ con las distancias reales entre todas las antenas.
- Restricciones que hay que respetar:
 1. Una antena j solo puede ser backup de i si la distancia entre ellas es menor a D ($d[i,j] < D$).
 2. Cada antena tiene que terminar con exactamente k backups asignados.
 3. Ninguna antena de la red puede ser el backup de más de b antenas en total.
 4. Una antena no puede hacerse backup a sí misma ($i \neq j$).
- Objetivo: El algoritmo tiene que devolver los conjuntos de backup de tamaño k para las n antenas en tiempo polinomial. Si por culpa de las distancias o los límites de capacidad es imposible cumplir todo a la vez, el programa debe avisar que no existe una solución posible.

3. Análisis del problema:

Si quisiéramos resolver esto probando todas las combinaciones posibles de antenas (fuerza bruta), el programa tardaría demasiado (tiempo exponencial) y no serviría para redes

grandes. El problema es que las decisiones están acopladas: si le asigno la antena j a la antena i , esa antena j ahora tiene menos "lugar" disponible para ayudar a otras antenas que quizás están lejos y no tienen otra opción.

Para resolverlo de forma inteligente y en tiempo polinomial, la mejor opción es modelarlo como un problema de Flujo Máximo en un Grafo Bipartito. La idea es "duplicar" cada antena: de un lado del grafo ponemos los nodos que *necesitan* backup (nodos de demanda) y del otro lado los nodos de las antenas que *ofrecen* el servicio (nodos de oferta).

Las restricciones del enunciado se traducen directo a capacidades en el grafo:

- Que cada una necesite k backups se modela limitando el flujo que sale de la fuente hacia los nodos de demanda.
- Que ninguna ayude a más de b se modela limitando el flujo que sale de los nodos de oferta hacia el sumidero.
- La distancia D nos sirve para crear o no las aristas intermedias.

Al usar Edmonds-Karp, nos aseguramos por teoría de grafos que la complejidad dependa únicamente de la cantidad de nodos y aristas, logrando el comportamiento polinomial que pide el trabajo.

4. Algoritmo:

4.1. Pseudocódigo:

```
Defino función buscar_conjuntos_backup(n: int, d: List<List>, D: float, b: int, k: int):
```

```
    Inicializo un grafo dirigido vacío G
    Declaro el nodo 'fuente' y el nodo 'sumidero'
```

```
    Itero con i desde 1 hasta n para crear la estructura base del grafo:
```

```
        Declaro nodo_A con el identificador "A_" + i
```

```
        Declaro nodo_B con el identificador "B_" + i
```

```
        Agrego al grafo G una arista desde 'fuente' hacia nodo_A con capacidad k
```

```
        Agrego al grafo G una arista desde nodo_B hacia 'sumidero' con capacidad b
```

```
    Itero con i desde 1 hasta n para evaluar las conexiones de demanda:
```

```
        Itero con j desde 1 hasta n para evaluar cada posible proveedora:
```

```
            Si i es distinto de j, entonces:
```

```
                Asigno a la variable distancia el valor de d[i-1][j-1]
```

```
                Si distancia es menor a D, entonces:
```

```
                    Agrego al grafo G una arista desde "A_" + i hacia "B_" + j
                    capacidad 1
```

```
    Invoco al algoritmo_edmonds_karp pasando el grafo G, 'fuente' y 'sumidero'
    Asigno a la variable valor_flujo el flujo total obtenido y a flujo_dict el mapa
    con los flujos por arista
```

```
    Declaro flujo_esperado con el resultado de n * k
```

```
    Si valor_flujo es menor a flujo_esperado, retorno None indicando que no hay
    solución posible
```

```

Inicializo una estructura de diccionario vacía resultado_backups
Itero con i desde 1 hasta n para reconstruir los conjuntos:
    Declaro nodo_A con el identificador "A_" + i
    Inicializo una lista vacía para resultado_backups[i]
    Itero por cada nodo_B vecino que esté conectado a nodo_A en flujo_dict:
        Asigno a flujo_enviado el valor del flujo en esa arista específica
        Si flujo_enviado es igual a 1, entonces:
            Extraigo el número identificador de la antenna desde el nombre de
            nodo_B
            Agrego ese identificador numérico a la lista de resultado_backups[i]

Retorno la estructura resultado_backups

```

4.4. Implementación:

```

import networkx as nx

def buscar_conjuntos_backup(n, d, D, b, k):
    """
    Encuentra el conjunto de backup para cada antenna utilizando Edmonds-Karp.

    Parametros:
    n (int): Cantidad de antenas.
    d (list of list): Matriz de distancias de n x n.
    D (float): Distancia máxima para que una antenna sea backup.
    b (int): Cantidad máxima de conjuntos de backup a los que puede pertenecer
    una antenna.
    k (int): Tamaño del conjunto de backup requerido para cada antenna.
    """

    G = nx.DiGraph()

    # Definimos nombres para la fuente y el sumidero
    fuente = 'fuente'
    sumidero = 'sumidero'

    # Construimos el Grafo
    for i in range(1, n + 1):
        # Nodo de necesidad (antenna i necesita backup)
        nodo_A = f"A_{i}"
        # Nodo proveedor (antenna i ofrece backup)
        nodo_B = f"B_{i}"

        # Conexión desde la fuente a los nodos A con capacidad k
        G.add_edge(fuente, nodo_A, capacity=k)
        # Conexión desde los nodos B al sumidero con capacidad b
        G.add_edge(nodo_B, sumidero, capacity=b)

    # Conexiones entre A y B basadas en la distancia

```

```

for i in range(1, n + 1):
    for j in range(1, n + 1):
        if i != j: # Una antena no suele ser backup de sí misma
            # Ajustamos índices para la matriz d (que es 0-indexed en Python)
            distancia = d[i-1][j-1]
            if distancia < D:
                G.add_edge(f"A_{i}", f"B_{j}", capacity=1)

# 2. Ejecución de Edmonds-Karp
# nx.maximum_flow utiliza Edmonds-Karp por defecto si se especifica la
# función.
valor_flujo, flujo_dict = nx.maximum_flow(G, fuente, sumidero,
    flow_func=nx.algorithms.flow.edmonds_karp)

# 3. Verificación de la solución
flujo_esperado = n * k
if valor_flujo < flujo_esperado:
    return None # No existe una solución posible que cumpla las restricciones

# 4. Reconstrucción de la solución
resultado_backups = {}
for i in range(1, n + 1):
    nodo_A = f"A_{i}"
    resultado_backups[i] = []

    # Revisamos a qué nodos B se les envió flujo desde A_i
    for nodo_B, flujo_enviado in flujo_dict[nodo_A].items():
        if flujo_enviado == 1:
            # Extraemos el número de la antena proveedora
            id_proveedora = int(nodo_B.split("_")[1])
            resultado_backups[i].append(id_proveedora)

return resultado_backups

```

4.5. Complejidad:

4.1. Complejidad Temporal:

Para determinar la complejidad temporal del algoritmo, analizamos el comportamiento de cada sección del código en Python en el peor de los casos, en función de la cantidad de antenas (n).

1. Construcción de la estructura base del grafo: El primer bucle *for* itera exactamente n veces. En cada iteración se calculan los identificadores de string y se ejecutan dos llamadas a `G.add_edge()`. La inserción de nodos y aristas en un grafo dirigido de *networkx* (*nx.DiGraph*) utiliza diccionarios internos de Python, por lo que toma tiempo constante $O(1)$. Esta etapa toma $O(n)$.
2. Mapeo de aristas según distancias: Consiste en un doble bucle *for* anidado que recorre todas las combinaciones posibles de antenas i y j . Esto realiza $n \times n = n^2$ iteraciones. En cada paso, accede a la matriz indexada en memoria ($d[i-1][j-1]$) en

tiempo $O(1)$. En el peor de los casos (si todas las antenas están a una distancia menor a D), se añade una arista por cada par mediante `G.add_edge()`. Por lo tanto, esta etapa toma $O(n^2)$.

3. Ejecución de Edmonds-Karp: La función `nx.maximum_flow` con el parámetro `flow_func=nx.algorithms.flow.edmonds_karp` ejecuta la implementación estándar de este algoritmo. Su complejidad teórica es $O(V \cdot E^2)$, donde $|V|$ es el número de vértices y $|E|$ el de aristas en el objeto G .
 - Vértices ($|V|$): El grafo contiene la fuente, el sumidero y dos nodos por cada antena (A_i y B_i). Por lo tanto, $|V| = 2n + 2$, lo que equivale a $O(n)$.
 - Aristas ($|E|$): En el peor escenario, hay n aristas desde la fuente, n aristas hacia el sumidero, y $n(n - 1) \approx n^2$ aristas intermedias que conectan los bloques. Así, $|E| \in O(n^2)$.
4. Sustituyendo estos valores en la cota superior del algoritmo de la librería:

$$O(|V| \cdot |E|^2) = O(n \cdot (n^2)^2) = O(n^5)$$

5. Reconstrucción de la solución: El bloque final inicializa el diccionario `resultado_backups` y corre un bucle externo n veces. Internamente, itera sobre los elementos devueltos por `flujo_dict[nodo_A].items()`. En el peor de los casos, un nodo A_i tiene $n-1$ vecinos en el diccionario de flujos. Las operaciones de strings (`split`) y la conversión a entero se hacen sobre cadenas de tamaño fijo, por lo que toman $O(1)$. Así, recorrer estas estructuras toma $O(n^2)$.

Conclusión de Complejidad Temporal:

Sumando el costo de todas las etapas, el tiempo total está dominado por la resolución del flujo máximo la complejidad temporal del algoritmo en el peor de los casos es $O(n^5)$, cumpliendo con el requisito de ser polinomial.

4.2. Complejidad Espacial:

El análisis de la memoria utilizada por las estructuras de datos de Python se calcula en relación con el tamaño de la entrada (n):

1. Matriz de distancias d : Almacena las distancias en una lista de listas de tamaño $n \times n$, consumiendo un espacio de $O(n^2)$.
2. Estructura del Grafo G : Un objeto `nx.DiGraph` representa las conexiones utilizando diccionarios de diccionarios (listas de adyacencia basadas en tablas de hash). El espacio requerido es proporcional a la cantidad de nodos más la cantidad de aristas. Como determinamos que en el peor de los casos $|V| \in O(n)$ y $|E| \in O(n^2)$, el grafo ocupa $O(n^2)$ de espacio.
3. Variables de salida de Flujo (`valor_flujo` y `flujo_dict`): La variable `valor_flujo` es un tipo de dato numérico simple ($O(1)$). Sin embargo, `flujo_dict` es un diccionario de diccionarios que guarda el valor del flujo de cada arista presente en G . Al tener la misma estructura de adyacencia que el grafo, consume $O(n^2)$ de memoria.
4. Estructura final `resultado_backups`: El diccionario final contiene n llaves (una por antena). Cada llave apunta a una lista de Python que almacena los identificadores enteros de sus backups asignados. Como cada lista puede guardar a lo sumo k

elementos, el espacio total es de nxk . Dado que por las condiciones del problema $k \leq n$, esta estructura está acotada por $O(n \cdot k) \subseteq O(n^2)$.

Conclusión de Complejidad Espacial:

Como la memoria principal del programa está destinada a mantener los diccionarios de adyacencia del grafo G y el mapa de flujos *flujo_dict*, la complejidad espacial total del algoritmo es de $O(n^2)$.

5. Ejemplo de ejecución y seguimiento:

- Parámetros iniciales: $n = 3$, $k = 1$, $b = 1$, $D = 5$
- Matriz de distancias d :

Antenas	Antena 1	Antena 2	Antena 3
Antena 1	0	3	4
Antena 2	3	0	4
Antena 3	4	4	0

Análisis de vecindad ($d[i][j] < D$ e $i \neq j$):

- La Antena 1 tiene como vecinos viables a la Antena 2 ($3 < 5$) y Antena 3 ($4 < 5$).
- La Antena 2 tiene como vecinos viables a la Antena 1 ($3 < 5$) y Antena 3 ($4 < 5$).
- La Antena 3 tiene como vecinos viables a la Antena 1 ($4 < 5$) y a la Antena 2 ($4 < 5$).

Paso 1: Construcción de la Red de Flujo (G)

El algoritmo genera un grafo dirigido bipartito añadiendo los nodos virtuales de demanda (**A**), de oferta (**B**), la fuente y el sumidero. Las aristas se crean con las capacidades parametrizadas:

- **Aristas desde la fuente:** (*fuelle*, A_1), (*fuelle*, A_2), (*fuelle*, A_3). Todas con *Capacidad* = 1 (k).
- **Aristas hacia el sumidero:** (B_1 , *sumidero*), (B_2 , *sumidero*), (B_3 , *sumidero*). Todas con *Capacidad* = 1 (b).
- **Aristas internas de decisión (Filtradas por distancia):**
 - (A_1 , B_2) con *Capacidad* = 1
 - (A_1 , B_3) con *Capacidad* = 1
 - (A_2 , B_1) con *Capacidad* = 1
 - (A_2 , B_3) con *Capacidad* = 1
 - (A_3 , B_1) con *Capacidad* = 1
 - (A_3 , B_2) con *Capacidad* = 1

Paso 2: Traza del Algoritmo de Flujo Máximo (Edmonds-Karp)

El algoritmo busca caminos de aumento desde la **fuelle** hasta el **sumidero** utilizando recorridos en anchura (BFS). Cada camino satura las aristas del trayecto:

1. **Camino de aumento 1:** *fuelle* $\rightarrow A_1 \rightarrow B_2 \rightarrow$ *sumidero*

- *Flujo enviado:* 1 unidad.
- *Estado:* Se saturan las capacidades de este camino. El nodo **A_1** completó su demanda.
- 2. **Camino de aumento 2:** *fuelle* → A2 → B3 → *sumidero*
 - *Flujo enviado:* 1 unidad.
 - *Estado:* Se saturan las capacidades de este camino. El nodo **A_2** completó su demanda.
- 3. **Camino de aumento 3:** *fuelle* → A3 → B1 → *sumidero*
 - *Flujo enviado:* 1 unidad.
 - *Estado:* Se saturan las capacidades de este camino. El nodo **A_3** completó su demanda.

Resultado de la ejecución: *valor_flujo* = 3

Paso 3: Estado Final de las Variables Internas

Al finalizar la ejecución del flujo máximo, el diccionario de flujos por aristas (***flujo_dict***) queda con el siguiente estado para las conexiones que transportan flujo activo:

Nodo Origen	Nodo Destino	Flujo Transportado	Capacidad Máxima	Estado de la Arista
fuelle	A_1	1	1	Saturada
fuelle	A_2	1	1	Saturada
fuelle	A_3	1	1	Saturada
A_1	B_2	1	1	Saturada
A_2	B_3	1	1	Saturada
A_3	B_1	1	1	Saturada
B_1	sumidero	1	1	Saturada
B_2	sumidero	1	1	Saturada
B_3	sumidero	1	1	Saturada

Paso 4: Verificación y Reconstrucción del Resultado

1. **Validación del flujo total:**
 - *flujo_esperado* = $n * k = 3 * 1 = 3$
 - Evaluación: *valor_flujo* (3) == *flujo_esperado* (3). La condición de fracaso no se cumple; el problema tiene una solución completa.
2. **Mapeo al diccionario de salida (*resultado_backups*):**
 El programa recorre los nodos de demanda **A_i** y extrae los destinos del bloque **B** que tengan *flujo_enviado* == 1:
 - **Para i = 1 (A_1):** Detecta flujo en la arista hacia **B_2**. Limpia el string y guarda el entero 2.

- **Para $i = 2$ (A_2):** Detecta flujo en la arista hacia B_3 . Limpia el string y guarda el entero 3.
- **Para $i = 3$ (A_3):** Detecta flujo en la arista hacia B_1 . Limpia el string y guarda el entero 1.

Estructura final retornada:

```
{
  1: [2],
  2: [3],
  3: [1]
}
```

6. Mediciones y validaciones empíricas:

Para evaluar el desempeño del algoritmo y contrastarlo con las hipótesis teóricas, se realizaron pruebas con instancias crecientes desde $n = 10$ hasta $n = 100$.

6.1. Registro de Tiempos de Ejecución

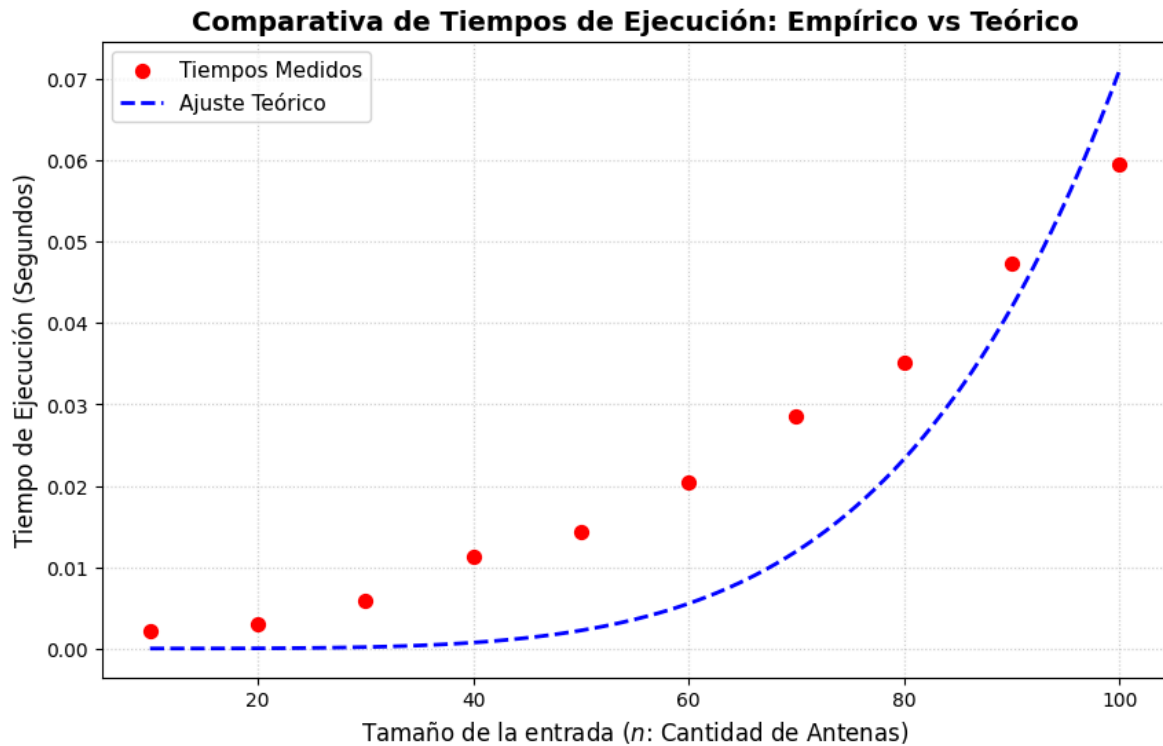
Los parámetros fijos de control asignados para mantener la homogeneidad de las pruebas fueron: distancia máxima admisible $D = 6.0$, cantidad de backups requeridos por antena $k = 2$ y capacidad límite de soporte por antena $b = 3$.

A continuación, se tabulan los tiempos medidos (en segundos) para cada tamaño de entrada analizado:

Tamaño de Entrada (n)	Tiempo de Ejecución [s]	Estado del Resultado
10	0.00208	Exitoso
20	0.00308	Exitoso
30	0.00580	Exitoso
40	0.01126	Exitoso
50	0.01438	Exitoso
60	0.02049	Exitoso
70	0.02854	Exitoso
80	0.03522	Exitoso
90	0.04726	Exitoso
100	0.05957	Exitoso

6.2. Análisis del Gráfico y Ajuste Curvilíneo

Utilizando el módulo de optimización de curvas (***curve_fit***), se proyectó la función de ajuste correspondiente a la cota teórica calculada para el algoritmo de Edmonds-Karp, cuya expresión responde a $f(n) = c \cdot n^5$.



Problema 3 - Algoritmos de aproximación:

1. Introducción:

En la presente sección se desarrolla el análisis y la resolución del problema de optimización combinatoria conocido como la suma de subconjuntos factibles, una variante del clásico problema Subset Sum.

Dado que encontrar la solución óptima a este problema en tiempo polinomial es computacionalmente inviable por pertenecer a la clase NP-Hard, se evalúa inicialmente un algoritmo Greedy, demostrando sus falencias y sub-optimalidad a través de un contraejemplo formal. Posteriormente, se propone y desarrolla un algoritmo de aproximación modificado, demostrando matemáticamente que garantiza devolver una solución cuya suma es, en el peor de los casos, al menos la mitad de la suma del subconjunto factible óptimo. Adicionalmente, se incluyen validaciones empíricas, ejemplos de seguimiento y un análisis exhaustivo de su complejidad temporal teórica contrastada con mediciones de rendimiento real.

2. Definición del problema y contraejemplo:

Se dispone de un conjunto de n enteros positivos $A = \{a_1, a_2, \dots, a_n\}$ y una cota superior representada por un entero positivo B . Se define como subconjunto factible a un subconjunto $S \subseteq A$ tal que la suma total de sus elementos sea estrictamente menor o igual al valor B . El objetivo principal es hallar un subconjunto factible S cuya sumatoria sea la máxima posible.

El planteo inicial propone evaluar el siguiente algoritmo secuencial para encontrar un subconjunto factible (no necesariamente óptimo):

```
subconjunto_factible(A, B)
  S={}
  T=0
  Para i=1,2,...,n
    Si T + A[i] <= B, entonces:
      S = S ∪ A[i]
      T = T + A[i]
  fin si
fin para
devolver S
```

2.1. El enfoque “ciego” y su caso de falla:

El algoritmo provisto posee una naturaleza Greedy "Ciega": recorre los elementos del conjunto A en su orden original de aparición, incorporando el elemento $A[i]$ al conjunto solución S de manera irrevocable siempre que el presupuesto disponible B lo permita.

Para demostrar la sub-optimalidad crítica de este enfoque, se presenta una instancia donde la suma devuelta por el algoritmo es estrictamente menor que la mitad de la suma de otro subconjunto factible existente.

Se propone la siguiente instancia de entrada:

- Conjunto: $A = \{3, 9, 1\}$
- Cota: $B = 10$

2.2. Seguimiento de la instancia:

El algoritmo recorre los elementos en el orden dado, evaluando la precondition de capacidad en cada iteración:

Iteración	Elemento $A[i]$	$T+A[i]$	$i \leq B?$	Acción	S actual	T actual
i=1	3	$0+3=3$	Si	Incorporar	{3}	3
i=2	9	$3+9=12$	No	Descartar	{3}	3
i=3	1	$3+1=4$	Si	Incorporar	{3,1}	4

El algoritmo finaliza y devuelve el subconjunto $S = \{3, 1\}$, cuya suma total es 4.

2.3. Demostración de la violación de garantía:

Es evidente que existe un subconjunto factible alternativo $S' = \{9, 1\}$ cuya suma resulta ser óptima para esta instancia:

$$suma(S') = 9 + 1 = 10 \leq B$$

Para que el resultado constituya un contraejemplo válido según los requerimientos del problema, debe cumplirse que la suma obtenida por el algoritmo sea estrictamente menor que la mitad de la suma de la solución óptima S' :

$$suma(S) < \frac{suma(S')}{2}$$

Reemplazando por los valores obtenidos:

$$4 < 5$$

La inecuación se cumple, confirmando la falla estructural del algoritmo. Esta deficiencia se origina por consumir capacidad del presupuesto de manera prematura con el primer elemento válido encontrado (en este caso, el 3), bloqueando irreversiblemente el ingreso de elementos subsiguientes de magnitud significativamente mayor (el 9).

3. Análisis del problema, supuestos y limitaciones:

3.1. Análisis de la falla y estrategia de mitigación

Como se evidenció en la sección anterior, la sub-optimalidad del algoritmo original radica en procesar los elementos de manera estrictamente secuencial según su orden de llegada, sin considerar su magnitud relativa. Este comportamiento Greedy Ciego provoca que elementos de bajo valor agoten el presupuesto B , imposibilitando la posterior inclusión de elementos de mayor peso que maximizarían la suma total.

La estrategia óptima para mitigar este comportamiento y garantizar una cota mínima de rendimiento consiste en añadir una etapa de preprocesamiento: ordenar el conjunto A de forma descendente. Al forzar al algoritmo a evaluar sistemáticamente los elementos de mayor a menor, se garantiza que el espacio disponible en B sea ocupado primero por los valores que más aportan a la sumatoria, transformando el enfoque ciego en un algoritmo Greedy Ordenado.

3.2. Supuestos y precondiciones

El diseño y la demostración de correctitud del algoritmo propuesto se fundamentan sobre las siguientes premisas matemáticas establecidas en el enunciado:

- Dominio de los elementos: El conjunto $A = \{a_1, a_2, \dots, a_n\}$ está compuesto estrictamente por números enteros positivos ($a_i > 0$ para todo i). Esto asegura que toda suma parcial T sea una función monótonamente creciente.
- Dominio de la cota: La capacidad máxima B es un número entero positivo ($B > 0$). Si $B = 0$, ningún elemento factible podría ser incluido, retornando lógicamente un conjunto vacío.

3.3. Comportamiento en casos límite

El diseño propuesto aborda intrínsecamente los casos límite sin requerir lógica condicional adicional:

- Conjunto vacío ($A = \emptyset$): El algoritmo no ejecuta iteraciones y retorna $S = \emptyset$. Dado que el óptimo también es 0, la garantía $0 \geq \frac{0}{2}$ se cumple trivialmente.
- Elementos que superan la cota ($a_i > B$): No es necesario filtrar el conjunto previamente. Si un elemento individual es mayor que B , la condición de validación $T + a_i \leq B$ lo descartará de manera natural.

4. Diseño del Algoritmo:

4.1. Descripción de la solución

El algoritmo `subconjunto_factible_aprox` recibe el arreglo A y la cota B , ordenando los elementos de A de mayor a menor en la memoria (in-place). Posteriormente, itera sobre los elementos, manteniendo una variable acumuladora T y un conjunto solución S . Si la suma del acumulador actual más el elemento evaluado no supera B , el elemento es aceptado irrevocablemente.

4.2. Demostración matemática de la garantía

Se requiere demostrar formalmente que la suma obtenida por nuestro algoritmo, a la cual llamaremos T , siempre es mayor o igual a la mitad de la suma del subconjunto factible óptimo, denotada como OPT . Es decir: $T \geq \frac{OPT}{2}$.

Para la demostración, el análisis se bifurca en dos escenarios posibles y exhaustivos:

- Caso 1: El algoritmo incorpora todos los elementos factibles evaluados.

Si ningún elemento con valor menor o igual a B es rechazado, esto implica que el algoritmo ha logrado incluir todos los elementos posibles dentro de la cota. Por definición, en este escenario, la suma obtenida es igual a la suma de todos los elementos factibles, lo cual es la cota máxima absoluta del problema. Por lo tanto, $T = OPT$, y la inecuación se cumple de forma trivial: $T \geq OPT \Rightarrow T \geq \frac{OPT}{2}$.

- Caso 2: El algoritmo rechaza al menos un elemento factible.

Sea a_k el primer elemento rechazado que por sí solo cumple $a_k \leq B$. (Si $a_k > B$), es matemáticamente imposible que pertenezca a cualquier subconjunto factible, incluido el óptimo, por lo que su rechazo no afecta la demostración).

Sea T_k la suma acumulada de los elementos en el conjunto S exactamente antes de evaluar a_k . La condición por la cual a_k es rechazado dicta que el presupuesto se excedería:

$$T_k + a_k > B \text{ (Ecuación 1)}$$

Dado que el arreglo A fue ordenado de mayor a menor en el preprocesamiento, sabemos con certeza que todos los elementos procesados (y aceptados) antes de evaluar a_k son mayores o iguales a él. Como a_k es el primer elemento válido rechazado, T_k debe contener, como mínimo, al primer elemento aceptado a_j (donde $j < k$). Por el ordenamiento, $a_j \geq a_k$. Por lo tanto, la sumatoria acumulada T_k es necesariamente mayor o igual al elemento rechazado:

$$T_k \geq a_j \geq a_k \Rightarrow T_k \geq a_k \text{ (Ecuación 2)}$$

Sumando la Ecuación 2 a la inecuación de la Ecuación 1:

$$T_k + T_k \geq T_k + a_k > B$$

$$2 \cdot T_k > B$$

$$T_k > \frac{B}{2} \text{ (Ecuación 3)}$$

Por otro lado, por la definición del problema, el subconjunto óptimo debe ser factible, lo que impone que su suma no puede exceder el presupuesto B :

$$OPT \leq B \Rightarrow \frac{OPT}{2} \leq \frac{B}{2} \text{ (Ecuación 4)}$$

Dado que el algoritmo es de naturaleza aditiva (nunca remueve elementos previamente aceptados), la suma final retornada T será como mínimo igual a T_k :

$$T \geq T_k \text{ (Ecuación 5)}$$

Por transitividad, encadenando las Ecuaciones 5, 3 y 4, obtenemos:

$$T \geq T_k > \frac{B}{2} \geq \frac{OPT}{2}$$

Concluyendo que $T > \frac{OPT}{2}$. Queda fehacientemente demostrado que el algoritmo garantiza un factor de aproximación estricto de $\frac{1}{2}$ para cualquier instancia del problema.

4.3. Pseudocódigo

El diseño se implementa de manera iterativa, optimizando el uso de la memoria dinámica.

```
Algoritmo subconjunto_factible_aprox(A, B):
  Ordenar A de mayor a menor

  S = {} // Inicialización del conjunto solución
  T = 0 // Acumulador de peso/valor

  Para i=1,2,...,n
    Si T + A[i] <= B entonces:
      S = S ∪ {A[i]}
      T = T + A[i]
    Fin Si
  Fin Para

  Retornar S
```

4.4. Estructuras de datos

Para lograr un diseño algorítmico eficiente tanto espacial como temporalmente, se determinan las siguientes estructuras:

- *A* (Arreglo/Lista secuencial): Se utiliza una lista de enteros en lugar de un conjunto matemático real, permitiendo su recorrido secuencial por índices y facilitando algoritmos de ordenamiento eficientes in-place o con memoria auxiliar controlada.
- *S* (Arreglo dinámico): Almacena los elementos seleccionados. Dado que la cantidad final de elementos en la solución es incierta, un arreglo de crecimiento dinámico (como un list en Python) amortiza el costo de inserción al final en tiempo constante $O(1)$.
- *T* (Variable escalar entera): Actúa como un caché de la sumatoria total del conjunto *T*. Al acumular aditivamente los valores en lugar de recalcular la suma de *S* en cada iteración, se evita añadir una complejidad interna $O(k)$, optimizando la verificación condicional a una operación aritmética de costo constante $O(1)$.

4.5. Implementación:

```
def subconjunto_factible_aprox(A: list, B: int) -> list:
    """
    Encuentra un subconjunto factible de A cuya suma no supere B.
    Garantiza que la suma resultante sea al menos la mitad de la solución óptima.
    """
    # 1. Preprocesamiento: Ordenar el arreglo de mayor a menor
    A_ordenado = sorted(A, reverse=True)

    S = [] # Conjunto solución
    T = 0 # Suma acumulada

    # 2. Procesamiento Greedy)
```



```

for elemento in A_ordenado:
    # Si el elemento cabe en el presupuesto restante, se incorpora
    if T + elemento <= B:
        S.append(elemento)
        T += elemento

return S

```

5. Seguimiento: Ejemplo de ejecución:

Para validar la mecánica del algoritmo propuesto subconjunto_factible_aprox, realizaremos una traza de ejecución utilizando exactamente la misma instancia que hizo fallar al algoritmo original en el caso de estudio (Sección 2).

- Conjunto: $A = \{3, 9, 1\}$
- Cota: $B = 10$

Paso 1: Preprocesamiento (Ordenamiento):

El algoritmo inicia ordenando el conjunto A de mayor a menor.

- Conjunto ordenado: $A = \{9, 3, 1\}$

Paso 2: Procesamiento Greedy:

Se inicializan las estructuras: conjunto $S = \emptyset$ y acumulado $T = 0$. Luego se itera sobre el arreglo ya ordenado:

Iteración	Elemento $A[i]$	$T+A[i]$	$i \leq B?$	Acción	S actual	T actual
i=1	9	$0+9=9$	Si	Incorporar	{9}	9
i=2	3	$9+3=12$	No	Descartar	{9}	9
i=3	1	$9+1=10$	Si	Incorporar	{9,1}	10

Resultado final:

El algoritmo finaliza su ejecución devolviendo el subconjunto $S = \{9, 1\}$ con una suma total de 10.

A diferencia del enfoque original (que había devuelto una suma de 4), el algoritmo de aproximación modificado logra alcanzar la solución óptima absoluta para esta instancia, demostrando empíricamente cómo la estrategia de priorización por magnitud maximiza el aprovechamiento de la capacidad B .

6. Análisis de Complejidad

Para determinar la eficiencia del algoritmo, dividimos su ejecución en las dos fases principales que lo componen: el ordenamiento inicial y el recorrido de los elementos. Asumimos que n es la cantidad de elementos en el conjunto A .

1. Fase de Ordenamiento:

La primera instrucción ordena el conjunto de entrada de mayor a menor. Al utilizar algoritmos de ordenamiento eficientes, esta operación tiene un costo computacional de $O(n \log n)$.

2. Fase de Procesamiento Greedy:

Posteriormente, el algoritmo utiliza un bucle for para iterar sobre los elementos ya ordenados. Este recorrido se realiza exactamente una vez (n iteraciones). Dentro del bucle, las únicas operaciones que se ejecutan son una suma, una comparación y, eventualmente, la inserción del elemento al final de la lista de soluciones. Todas estas son operaciones elementales de tiempo constante $O(1)$. Por lo tanto, el costo total de esta iteración es estrictamente lineal $O(n)$.

Complejidad Total:

El tiempo total de ejecución es la suma de ambas fases:

$$T(n) = O(n \log n) + O(n)$$

En el análisis asintótico, el término de mayor orden domina el tiempo de ejecución a medida que n crece. Por consiguiente, el costo del bucle lineal es absorbido por el costo del ordenamiento. La complejidad temporal final del algoritmo es:

$$T(n) = O(n \log n)$$

7. Set de datos:

Mediante un algoritmo de python generamos una serie reproducible (`random.seed(42)`) de datos para probar el nuevo algoritmo de aproximación. Para cada tamaño N , generamos un arreglo de valores enteros aleatorios entre 1 y 1.000, y una cota B que es aproximadamente el 40% de la suma total de los valores.

Los resultados de cada prueba fueron:

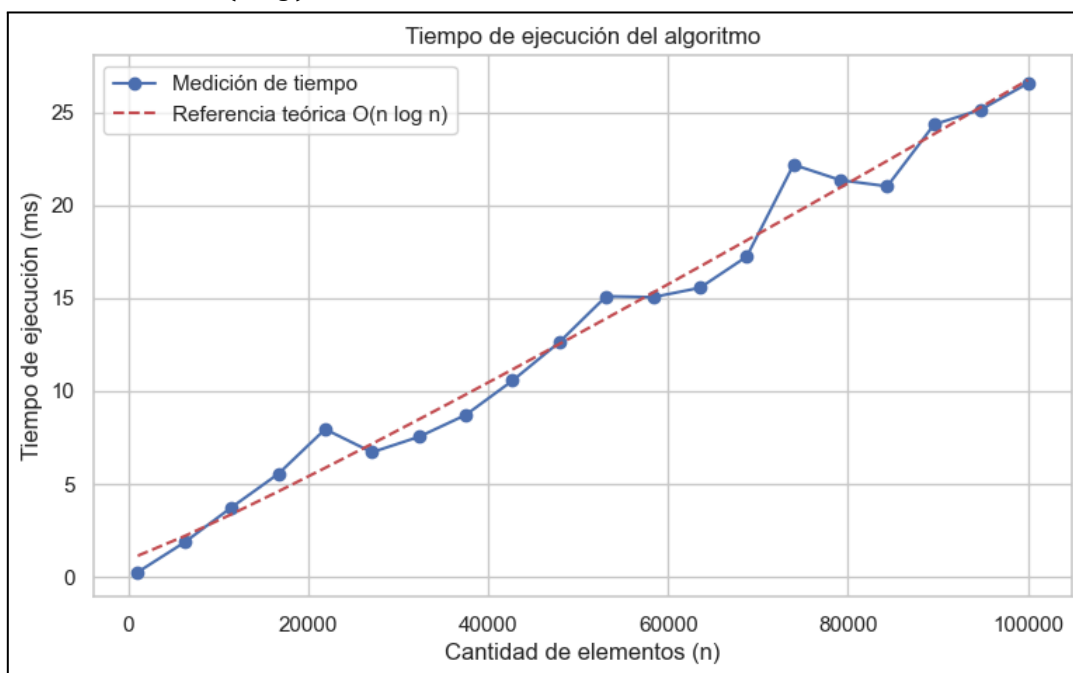
Caso	N	B	Suma Obtenida	Tiempo
Contraejemplo	3	10	10	0.0066 ms
100	100	18112	18106	0.0172 ms
500	500	204574	204574	0.0690 ms
1000	1000	2009388	2009388	0.1371 ms
5000	5000	98286	98286	0.8752 ms
10000	10000	1003147	1003147	1.6381 ms
50000	50000	10001522	10001522	9.8017 ms

8. Validaciones empíricas:

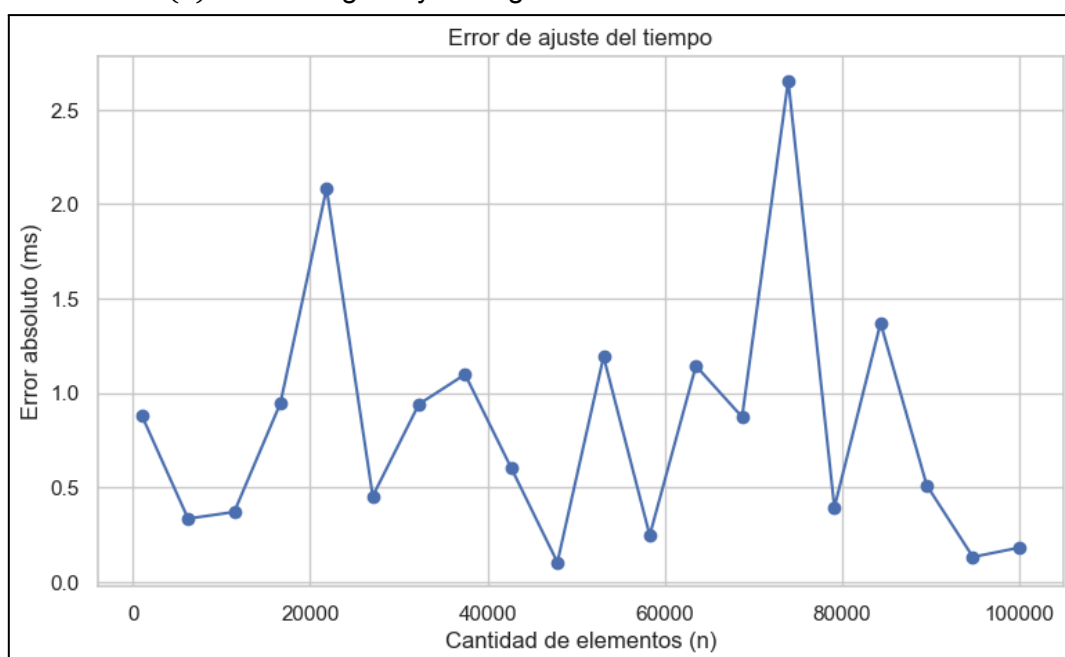
Se realizaron mediciones exhaustivas utilizando diferentes cantidades de elementos. Para simular fielmente el escenario del problema, en cada instancia se instanció un arreglo numérico y se utilizó la biblioteca random (con una semilla fija para garantizar la reproducibilidad) para asignar pesos arbitrarios a los elementos y establecer una cota máxima B proporcional al peso total.

Para evaluar el algoritmo bajo un volumen de datos significativo, cada conjunto de pruebas contiene arreglos con tamaños n que varían progresivamente desde 100 hasta 100.000 elementos, abarcando distintos puntos de medición. En cada uno de estos puntos se registró el tiempo de ejecución computacional neto.

Posteriormente, se realizó un ajuste por mínimos cuadrados utilizando la biblioteca scipy y se calculó el error de ajuste para contrastar los resultados empíricos contra la complejidad teórica demostrada $O(n \log n)$.



Como se observa en el gráfico, la curva de tiempo exhibe un crecimiento que se ajusta perfectamente al modelo asintótico $O(n \log n)$. El ajuste demuestra que el tiempo total de ejecución está dominado estrictamente por la etapa de ordenamiento inicial, confirmando la eficiencia lineal $O(n)$ del bucle greedy subsiguiente.



El gráfico de error de ajuste refleja variaciones absolutas marginales entre el tiempo medido y la proyección teórica. Las ligeras oscilaciones presentes en la serie son propias del entorno de ejecución, pero la ausencia de una divergencia cuadrática corrobora de manera concluyente el modelo de complejidad establecido en la sección analítica.

9. Conclusión:

En el presente trabajo se abordó el desafío de encontrar un subconjunto factible cuya suma máxima no supere una cota B , analizando el problema desde la perspectiva de los algoritmos de aproximación.

En primera instancia, el análisis del enfoque greedy ciego provisto originalmente demostró que procesar los elementos de un conjunto en orden arbitrario resulta en una estrategia subóptima. Mediante un contraejemplo formal, se expuso cómo la inclusión prematura de elementos de bajo valor agota el presupuesto disponible, bloqueando de manera irreversible el ingreso de elementos de mayor peso y produciendo resultados que caen muy por debajo del óptimo.

Para subsanar esta deficiencia geométrica en la toma de decisiones, se propuso y desarrolló el algoritmo subconjunto_factible_aprox. La solución introduce una fase de ordenamiento previo (de mayor a menor magnitud), transformando la lógica en un algoritmo Greedy Ordenado.

¿Se cumple con la garantía solicitada?

Sí. Como se demostró formalmente en la Sección 4.2, priorizar la evaluación de los elementos de mayor a menor asegura un factor de aproximación estricto de $\frac{1}{2}$. Para cualquier instancia posible, el algoritmo garantiza que la suma obtenida siempre será mayor o igual a la mitad del óptimo ($T \geq \frac{OPT}{2}$). Además, las validaciones empíricas confirmaron que esta certeza matemática se alcanza manteniendo una alta eficiencia computacional, operando en un tiempo asintótico de $O(n \log n)$ y consolidando al algoritmo como una solución escalable y robusta.

Problema 4 - Algoritmos aleatorios:

1. Introducción:

En esta última sección del trabajo práctico se buscará realizar el desarrollo de un algoritmo aleatorio en tiempo polinomial, propuesto por la cátedra. Dicho esto, luego de encontrar el algoritmo resuelto en tiempo polinomial, se realizará un análisis del mismo, viendo su diseño, complejidad, seguimiento y pruebas para evaluar los resultados, y ver si coinciden con el valor esperado mínimo.

2. Definición del problema:

Se plantea una contraparte del problema de 3-coloreo de un grafo G , donde en vez de verificar si todos los vértices de un grafo pueden colorearse de 3 colores distintos tal que no hayan dos vértices adyacentes pintados con un mismo color, se plantea buscar maximizar la cantidad de aristas satisfechas, denominada c^* , tal que para cada arista (u, v) , se considera que una arista se encuentra satisfecha si los vértices u y v están pintados de distinto color.

Dado el planteo, se busca desarrollar un algoritmo que resuelva el problema de optimización de 3-coloreo, cuyo número esperado de aristas satisfechas sea de $2/3 \cdot c^*$.

3. Análisis del problema:

La definición teórica del problema es la siguiente:

Dado un grafo $G = (V, E)$, y 3 colores que pueden corresponder al conjunto $C = [0, 1, 2]$, se debe asignar cada color a una arista $(u, v) \in E$ tal que $color(u) \neq color(v)$, siendo esta la condición para obtener una arista satisfecha.

El problema es buscar calcular el valor de c^* tal que este representa la cantidad de aristas satisfechas de G .

Este problema puede resolverse con un algoritmo randomizado, cuyos detalles de la resolución se encuentran en la siguiente sección.

4. Algoritmo:

4.1. Formulación del algoritmo aleatorio:

Para cada vértice $v \in V$, se tiene que:

- Elegir de manera aleatoria uno de los colores del conjunto $C = [0, 1, 2]$.
- Todos los colores tienen la misma probabilidad de salir, siendo $p_c = 1/3$.
- La selección del color actual no influye en la elección del siguiente color.

Dado el problema, si un vértice está pintado de un color tal que: $color(v) = c$, entonces la probabilidad de que una arista sea satisfecha es:

$$p_{satisfecha} = 1 - p_c = 1 - \frac{1}{3} = \frac{2}{3}$$

Mientras que la probabilidad de que no sea satisfecha será:

$$p_{no_satisfecha} = 1 - p_{satisfecha} = 1 - \frac{2}{3} = \frac{1}{3}$$

4.2. Variable Indicadora:

Una vez dada la formulación del problema, se debe definir la variable indicadora o discreta para determinar la cantidad esperada de las aristas satisfechas (valor esperado). Esto se puede modelar como una variable Bernoulli, en donde:

$$\begin{aligned} X_e &= 1 \text{ si la arista está satisfecha} \\ X_e &= 0 \text{ si la arista no está satisfecha} \end{aligned}$$

Definimos la suma total de aristas satisfechas como:

$$X = \sum_{e \in E} X_e$$

4.3. Cálculo del valor esperado:

El enunciado nos pide calcular $E[X]$:

$$E[X] = E\left[\sum_{e \in E} X_e\right]$$

La esperanza para cada X_e equivale a la esperanza de una variable Bernoulli:

$$E[X_e] = P(X_e = 1) = p_{satisfecho} = \frac{2}{3}$$

Aplicando la linealidad, la esperanza de X vendrá dada por

$$E[X] = E\left[\sum_{e \in E} X_e\right] = \sum_{e \in E} (E[X_e]) = \sum_{e \in E} \frac{2}{3} = |E| \cdot \frac{2}{3}$$

Donde $|E|$ representa la cantidad de aristas del grafo G .

Entonces, para el c^* definido con anterioridad, en caso de que todos los vértices $v \in V$ esten pintados con colores distintos, se debe cumplir que:

$$|E| \geq c^* \Rightarrow E[X] = \frac{2}{3} \cdot |E| \geq \frac{2}{3} \cdot c^*$$

4.4. Estructuras utilizadas:

Las estructuras de datos utilizadas para el algoritmo son las siguientes:

- Grafo: Estructura principal del algoritmo, representado como una lista de adyacencia.
- Colores: Diccionario cuyas claves son los vértices del grafo, y su valor es el color asignado entre 0 y 2 inclusive.
- aristas_visitadas: Lista de aristas visitadas para no volver a recorrerla.

4.5. Pseudocódigo

```
// Función aleatoria que colorea los vertices de un grafo
def colorear_3_colores(Grafo)
    colores = {}
    Para cada vertice en Grafo:
        colores[vertice] = random(0, 2)

    retornar colores

//Se calcula la cantidad de aristas satisfechas para un grafo colorado de forma
aleatoria.
def calcular_aristas_satisfechas(Grafo):
    vertices_coloreados = colorear_3_colores(Grafo):
    contador_vertices = 0
    aristas_visitadas = []

    Para cada u en Grafo:
        Para cada v en Adyacentes(u);
            Si arista(u,v) no esta en aristas_visitadas:
                aristas_vistadas.agregar((u, v))

            si vertices_coloreados[u] != vertices_coloreados[v]:
                contador_vertices ++

    retornar contador_vertices
```

4.6. Implementación:

```
import random

def colorear_3_colores(Grafo):
    colores = {}
    for vertice in Grafo:
        colores[vertice] = random.randint(0, 2)
    return colores

def calcular_aristas_satisfechas(Grafo):
    vertices_coloreados = colorear_3_colores(Grafo)
    contador_vertices = 0
    aristas_visitadas = []

    for u in Grafo:
        for v in Grafo[u]:
            if (v, u) not in aristas_visitadas:
                aristas_visitadas.append((u, v))
            if (vertices_coloreados[u] != vertices_coloreados[v]):
                contador_vertices += 1

    return contador_vertices
```

4.7. Complejidad:

A continuación se realizará el análisis de la complejidad temporal del algoritmo randomizado junto con el contador de aristas satisfechas, realizando el seguimiento de las estructuras de control utilizando la notación de Big O.

4.7.1. Complejidad Temporal:

La función **colorear_3_colores(Grafo)** simplemente cuenta con un ciclo for para recorrer un grafo de n vértices, por lo que la complejidad de la función será de $O(|V|)$ con $|V|$ como la cantidad de vértices del grafo.

La función **calcular_aristas_satisfechas(Grafo)** consta de dos ciclos for anidados y dos ifs consecutivos.

- El primer ciclo for recorre todos los vértices u de G , teniendo un rango de $|V|$ vértices, por lo que su complejidad será de $O(|V|)$.
- El segundo ciclo for se encuentra anidado al anterior ciclo, recorriendo todos los vértices adyacentes del vértice u . Notar que este ciclo depende de la cantidad de vértices adyacentes que tenga u , que es el equivalente a la cantidad de aristas pertenecientes a E , por lo que su complejidad será $O(|E|)$.
- Cada if se realiza en tiempo constante, por lo que su complejidad es de $O(1)$.

Los ciclos for anidados al depender de los vértices y aristas que conforman al grafo, tendrán una complejidad de $O(|V| + |E|)$, siendo esta la complejidad del algoritmo.

5. Ejemplo de ejecución y seguimiento:

Supongamos que se cuenta con el siguiente grafo G de prueba, representado como una lista de adyacencia:

```
Grafo = {  
  1: [2, 3, 4],  
  2: [3, 4],  
  3: [1, 2, 4],  
  4: [2, 3],  
}
```

El conjunto de aristas para este grafo es el siguiente:

$$E = \{(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)\}$$

Primero se debe asignar uno de los tres colores (número) de manera aleatoria llamando a la función **colorear_3_colores(Grafo)**, por lo que un diccionario posible de colores asignados que retorna la función es: $E = \{1: 1, 2: 1, 3: 0, 4: 2\}$

A partir de este momento, se realizará el seguimiento puntual de cada vértice para ver cómo se va realizando el conteo de aristas satisfechas.

Iteraciones:

```

u = 1
v = Grafo.obtener_adyacentes(u) = 2
v not in visitados => True
aristas_visitadas.add(1, 2)
vertices_coloreados[1] != vertices_coloreados[2] => False
contador_vertices = 0 (sigue en 0)

u = 1
v = Grafo.obtener_adyacentes(u) = 3
v not in visitados => True
aristas_visitadas.add(1, 3)
vertices_coloreados[1] != vertices_coloreados[3] => True
contador_vertices = 1

u = 1
v = Grafo.obtener_adyacentes(u) = 4
v not in visitados => True
aristas_visitadas.add(1, 4)
vertices_coloreados[1] != vertices_coloreados[4] => True
contador_vertices = 2

u = 2
v = Grafo.obtener_adyacentes(u) = 3
v not in visitados => True
aristas_visitadas.add(2, 3)
vertices_coloreados[2] != vertices_coloreados[3] => True
contador_vertices = 3

u = 2
v = Grafo.obtener_adyacentes(u) = 4
v not in visitados => True
aristas_visitadas.add(2, 4)
vertices_coloreados[2] != vertices_coloreados[4] => True
contador_vertices = 4

u = 3
v = Grafo.obtener_adyacentes(u) = 1
v not in visitados => True
aristas_visitadas.add(3, 1)
vertices_coloreados[3] != vertices_coloreados[1] => False
contador_vertices = 4 (Sigue igual)

u = 3
v = Grafo.obtener_adyacentes(u) = 2
v not in visitados => True
aristas_visitadas.add(3, 2)
vertices_coloreados[3] != vertices_coloreados[2] => False
contador_vertices = 4 (Sigue igual)

u = 3

```

```

v = Grafo.obtener_adyacentes(u) = 4
v not in visitados => True
aristas_visitadas.add(3, 4)
vertices_coloreados[3] != vertices_coloreados[4] => True
contador_vertices = 5

u = 4
v = Grafo.obtener_adyacentes(u) = 2
v not in visitados => True
aristas_visitadas.add(4, 2)
vertices_coloreados[4] != vertices_coloreados[2] => False
contador_vertices = 5 (Sigue igual)

u = 4
v = Grafo.obtener_adyacentes(u) = 3
v not in visitados => True
aristas_visitadas.add(4, 3)
vertices_coloreados[4] != vertices_coloreados[3] => False
contador_vertices = 5 (Sigue igual)

retorno 5

```

6. Mediciones:

En esta sección muestra el análisis de las pruebas realizadas para verificar si los resultados obtenidos corresponden a la demostración teórica.

6.1. Metodología de pruebas:

Para realizar las pruebas, se optó por utilizar un grafo distrito al utilizado en el seguimiento. el cual, consta de más vértices y aristas para obtener resultados más realistas. A su vez, se crearon dos archivos **test.py** y **crear_mediones.py** para realizar el testeo con el grafo un total de 10000 veces y crear el set de datos utilizado para el gráfico.

6.2. Gráfico comparativo:

Se optó por este gráfico de conteo de ocurrencias debido a que los resultados obtenidos se encuentran en un rango cuyo dominio es continuo y no discreto.

El grafo que se utilizó para realizar los testeos es el siguiente:

```

Grafo = {
    1: [2, 3, 4],
    2: [1, 3, 5],
    3: [1, 2, 4, 5, 6],
    4: [1, 3, 6],
    5: [2, 3, 6],
    6: [3, 4, 5]
}

```

Para este Grafo, se tiene que la cantidad de aristas $|E| = 10$, por lo que el valor medio estimado de aristas satisfechas es de $E[X] = 10 * \frac{2}{3} = 6.667$

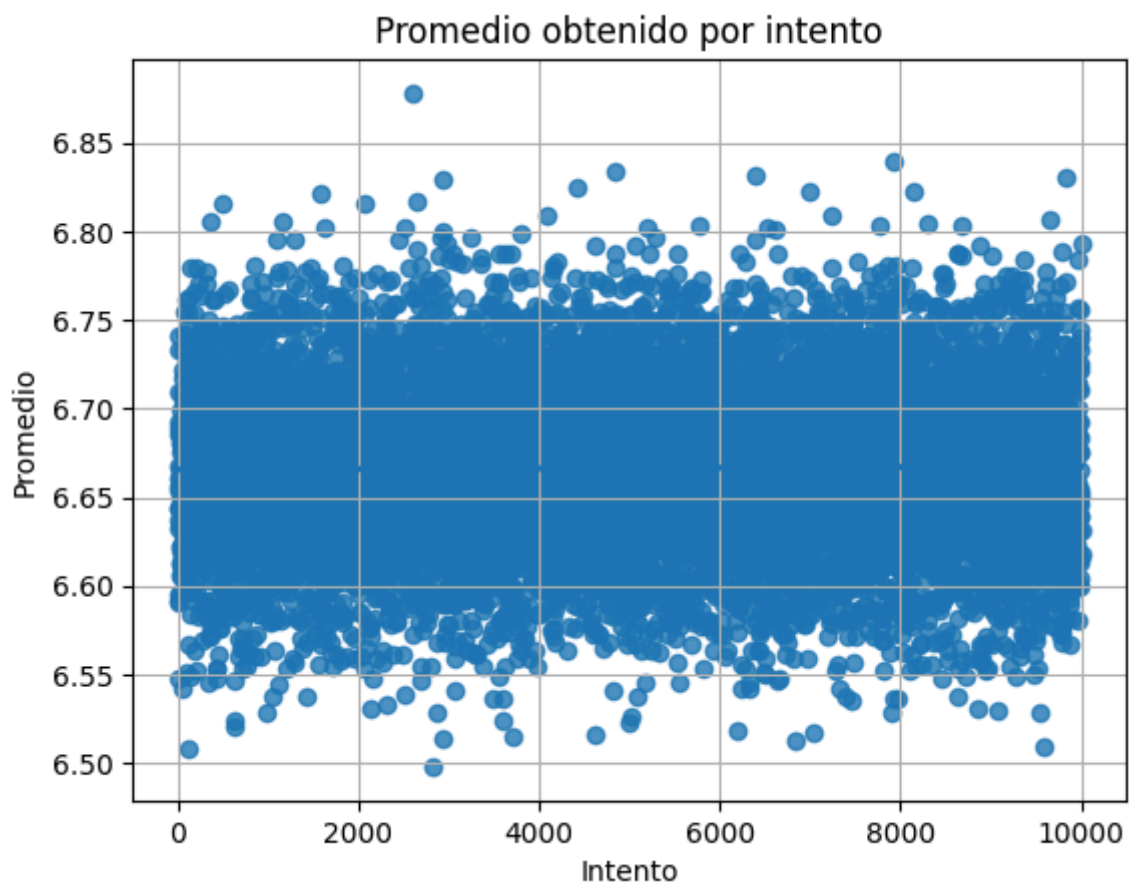


Imagen: Promedio vs Intentos

Como puede observarse en el gráfico, la mayoría de los promedios obtenidos se encuentran en el intervalo $[6.60, 6.75]$ y solo hay unos pocos valores que se encuentran cerca de los extremos del intervalo $[6.50, 6.85]$, por lo que la mayoría de las pruebas realizadas dan alrededor de la media de la cantidad de aristas satisfechas.

7. Conclusión:

Luego de haber realizado las pruebas requeridas y analizar los resultados obtenidos, se observó que efectivamente el valor de la cantidad de aristas satisfechas tiende a aproximarse al valor medio esperado, siendo este de $|E| * \frac{2}{3}$.

Dicho esto, a través de los resultados experimentales se puede concluir con que el algoritmo cumple con la garantía de que el valor esperado es de al menos $\frac{2}{3} * c * \text{aristas}$, cumpliendo esto en tiempo polinomial.